The Performance Profiling of `pipe()`

Arjun Anil, Aidan Liaw
aanil3@wisc.edu, aidan.liaw@student.uq.edu.au

1 Introduction

The POSIX standard defines `pipe()` as a function that creates an anonymous pipe and places two file descriptors, for the read and write ends, in its argument [10]. Through the Virtual File System a pipe is used with `read()` and `write()` [4, 12], and its kernel buffers are guarded by a mutex. Two sizes matter: `PIPE_BUF`, up to which a write is atomic, i.e. never interleaved with another writer's data [5], and the finite pipe capacity, past which a blocking producer sleeps until a consumer drains the pipe. This paper analyses `pipe()`: we measure its latency and throughput and use `perf`, `strace` and `ftrace` to explain where the time goes.

2 Experiment

2.1 Variables

- The message size, as suggested by the instructor: 4B, 16B, 64B, 256B, 1024B, 4096B, 16384B, 65536B, 262144B and 524288B. 4096B matches `PIPE_BUF`, which is preset [11], and 65536B the pipe capacity `PIPE_LENGTH`, which we leave at the kernel default.
- The number of measurements: 20 warmup and 2000 timed round trips per size in each of 5 runs (latency); 3 transfers of 64 MiB per size in each of 5 runs (throughput; 4B and 16B are capped at 2,000,000 chunks).
- Both processes run under `SCHED_FIFO` at priority 99, the parent on CPU0, the child on CPU1 and the profiler on CPU2, with the frequency pinned to 4 GHz.
- Blocking I/O: no `O_NONBLOCK`, so tasks sleep instead of spinning.

2.2 Hypotheses

1. The variance in data measurements between runs should be minimal, given the care taken to ensure process isolation.
2. Performance will get worse for data sizes above `PIPE_BUF`, which we expected might split a write into several operations, and for data sizes above `PIPE_LENGTH`, as the producer must stall until the consumer drains the pipe.

2.3 Apparatus

The laptop has an Intel Core Ultra 7 258V and runs Fedora Linux 44, kernel 7.1.8; only its four P-cores are used.

Timer. Timer resolution is the smallest non-zero difference between back-to-back reads. `gettimeofday()` resolves 1 μs and `CLOCK_MONOTONIC_COARSE` 1 ms, too coarse for a message of about a microsecond, while `CLOCK_MONOTONIC` (11 ns) and `rdtsc` (7.3 ns) are not. We use `rdtsc`, whose counter is invariant on this CPU and read in the parent only, so each latency is a difference of two reads on one core; cycles are converted to time by calibrating against `CLOCK_MONOTONIC` (3.302 GHz). Around a `sleep(2)` the three fine timers agree to within 10.5 μs, and `gettimeofday()`, the less accurate reference, is within 1.9 μs of `CLOCK_MONOTONIC`.

Procedure. For latency, a parent and a child (`fork()`) are connected by two pipes and the child echoes each message; the one-way latency is half the round trip, as the counters of two cores cannot be compared. For throughput the parent streams chunks and the child returns one acknowledgement byte after the whole transfer. Noise can only add time, so we report the minimum latency and the best throughput [1, 8]. The profilers slow the benchmark, so they run separately and only counts and proportions are used: `perf` [2, 7], `strace` [6] and `ftrace`, which also sees the kernel calls `strace` cannot [3, 9].

3 Results

Figure 1 shows that a small message takes about 1 μs one way (0.98 μs at 4B), stepping up to 2.12 μs at 4KB, and beyond the pipe capacity latency is roughly proportional to the size (113.9 μs at 512KB). Throughput rises from 21 MiB/s (4B) to a peak of 4,742 MiB/s at 16KB, is 25% lower at 64KB (3,578 MiB/s) and stays there up

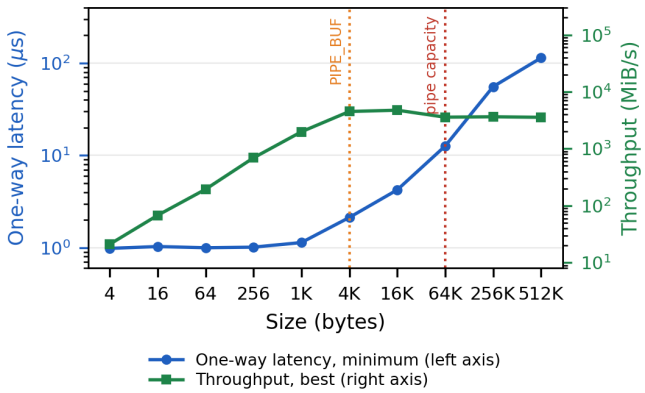


Figure 1: Minimum one-way latency (left axis) and best throughput (right axis) versus transfer size; dotted lines mark `PIPE_BUF` and the pipe capacity.

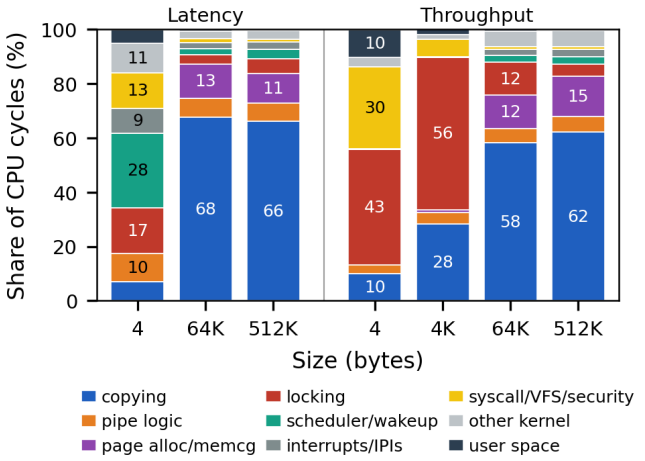


Figure 2: Share of the CPU cycles per component, for latency and throughput.

to 512KB. The minima reproduced to within 0.9–9.7% (latency) and 5.8–14% (throughput, chunks of 1KB and larger; 29–39% for 4B and 16B) across five runs.

4 Where the Time Goes

`strace` shows one `write()` per message at every size. A round trip of up to 64KB causes 2 context switches, 2 wakeups and 2 inter-processor interrupts; 256KB and 512KB cause 14 and 30 context switches, i.e. $2(2n-1)$ for n pipe capacities, because the writer fills the pipe and sleeps until the reader drains it. For 4B the median round trip is 3.06 μs, but the tasks run for only 2.11 μs: for 0.95 μs (31%) neither runs, as a sleeping core is woken.

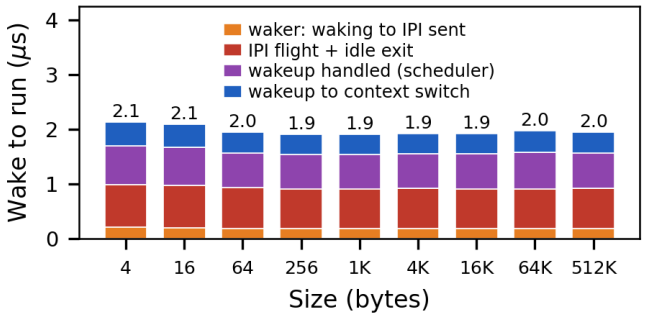


Figure 3: Phases of a wakeup (median, latency experiment, traced; 256KB omitted, as only 43% of its wakeups found the core idle).

In Figure 2, for 4B no component dominates: scheduling and wakeup take 27.5%, locking 16.8% and copying only 7.1%. At 64KB and 512KB copying takes 68% and 66%. Waking to running takes 1.9–2.1 μs at every size (Figure 3): a fixed cost paid twice per round trip. The `ftrace` call tree agrees: at 4B reads are dominated by the blocked `schedule`, writes by the wakeup chain.

Streaming works as a pipeline: for chunks of 64B to 16KB there are almost no context switches and 1.9–2.0 cores are busy (CPU time over wall time). From 64KB each chunk causes 2 to 16 context switches and only 0.86–0.91 cores are busy, as the writer fills the pipe and sleeps. The cost per KiB even falls (1,741 to 991 cycles), but with half the parallelism throughput drops. At 4KB the tasks spin on the pipe mutex (56% locking).

5 Discussion and Conclusion

The first hypothesis holds only in part: the minima reproduced within 10–14%, but the median small-message round trip is 56% above the minimum. Of the second, the `PIPE_BUF` half is not supported: every message is a single `write()` and throughput peaks at 16KB instead of falling at 4KB. Latency does step up at 4KB (the median round trip grows by 2.1 μs from 1KB, half of it with neither task running), which we cannot explain. The capacity half is supported: past 64KB the writer must sleep and throughput falls by 25%.

Small messages are therefore a scheduling problem and large messages a copying problem. These results come from a single laptop with the frequency pinned at 4 GHz. Two further caveats are that `SCHED_FIFO` and tracing add overhead of their own, and that we did not vary the pipe capacity or page size.

References

[1] Gernot Heiser. 2026. Systems Benchmarking Crimes. Accessed: 2026-09-21. https://www.cse.unsw.edu.au/~gernot/benchmarking-crimes.html

[2] Linux kernel community. 2026. Linux kernel profiling with `perf`. Accessed: 2026-09-21. https://perf.wiki.kernel.org/index.php/Main_Page

[3] Linux Kernel Organization. 2026. ftrace — Function Tracer. Accessed: 2026-09-21. https://docs.kernel.org/trace/ftrace.html

[4] Linux Kernel Organization. 2026. Overview of the Linux Virtual File System (VFS). https://docs.kernel.org/filesystems/vfs.html

[5] Linux man-pages project. 2024. pipe(7) — Linux Manual Page. https://man7.org/linux/man-pages/man7/pipe.7.html

[6] Linux man-pages project. 2024. strace(1) — Linux Manual Page. https://man7.org/linux/man-pages/man1/strace.1.html

[7] Red Hat Customer Portal. 2025. How do I use raw events with perf? Accessed: September 21, 2026. https://access.redhat.com/articles/7129145

[8] James E. Smith. 1988. Characterizing Computer Performance with a Single Number. *Commun. ACM* 31, 10 (1988), 1202–1206. doi:10.1145/63039.63043

[9] Stack Overflow contributors. 2019. Difference between ftrace and strace in Linux. Accessed: 2026-09-21. https://stackoverflow.com/questions/58047458/difference-between-ftrace-and-strace-in-linux

[10] The IEEE and The Open Group. 2024. *The Open Group Base Specifications Issue 8, IEEE Std 1003.1-2024*. Technical Report. The Open Group. https://pubs.opengroup.org/onlinepubs/9799919799/

[11] Linus Torvalds and Linux Kernel Contributors. 2026. Linux Kernel Source Code: `include/uapi/linux/limits.h`. https://github.com/torvalds/linux/blob/master/include/uapi/linux/limits.h

[12] Unknown. 2011. Linux Kernel Implementation of Pipes and FIFOs. https://www.slideshare.net/slideshow/linux-kernel-implementation-of-pipes-and-fifos/872826